CHAPTER 6: REFERENCE ARCHITECTURES AND RETRIEVAL AGAINST LIVE STATE

Summary: *Retrieval is where a design most often looks correct and returns wrong answers with a straight face. Two mistakes account for nearly all of it: forcing one matching strategy onto traffic that contains two incompatible query shapes, and pointing a retrieval pipeline at data that changes faster than the index does. Both produce fluent, cited, confident output. This chapter covers how to match a strategy to a data shape, why live state does not belong in an index, and what grounding actually guarantees.*

Two Query Shapes With Opposite Requirements

The single most useful retrieval insight is that a corpus does not have one query pattern. It usually has two, and they want opposite things.

Identifier lookups. A human types an order number, a part code, an error string, a policy reference. These need exact matching. The user knows precisely what they are asking for, and near-misses are worthless: an answer about a similar part number is not a partially correct answer, it is a wrong one.

Conceptual questions. Somebody asks how a process works, or what the policy says about a situation, using words that appear nowhere in the document that answers them. These need semantic matching, because the whole point is to find material that means the same thing in different vocabulary.

Now the part that decides the answer. **Semantic embedding actively damages identifier lookups**, because embedding is a compression that keeps meaning and discards surface form, and an identifier has no meaning to keep. Two adjacent part numbers embed to nearly the same place. Meanwhile exact matching is useless on conceptual questions, because the words that would match are not there.

So when a scenario describes traffic containing both shapes, the **answer preserves both matching strategies and routes between** them. Picking one strategy for all traffic is not a trade-off, it is choosing which half of your users to fail.

The **named anti-pattern is single-strategy retrieval on mixed traffic**. It reaches production because the evaluation set was written by somebody thinking about one query shape, and the other half of the traffic never appears in it.

Data and query shape	What matches it	What breaks it
Exact identifiers, codes, references	Lexical or exact matching, or a direct lookup by key	Embedding, which blurs identifiers into their neighbours
Conceptual questions in the user's own words	Semantic similarity over embedded chunks	Exact matching, since the matching words are absent
Both, in the same traffic	Keep both, and route each query to the one that fits	Choosing one and calling it a simplification
Facts that change between indexing and asking	A tool call to the system that owns the fact	Any index, however often it is rebuilt

Live State Does Not Belong in an Index

Name the failure: **retrieval applied to live state** is pointing an index at data that changes faster than the index is rebuilt, and it produces the worst output shape available, which is a confident, well-sourced, stale answer.

The mechanics are unforgiving. An index is a snapshot. Between the snapshot and the question, inventory moves, an order ships, a balance changes, a ticket closes. The retrieval succeeds. The passage it found is real. The answer is wrong, and it arrives with a citation attached that makes it look more trustworthy than an uncited guess.

The test is the one from the pattern chapter, pointed at data: how fast does **this change**, compared with **how often the index is rebuilt**? If the answer is "faster", no

rebuild schedule fixes it, because the problem is the architecture rather than the cadence.

Live state belongs behind a call to the system that owns it. The **system of record** answers **as of now**; an **index** answers **as of whenever**. For a stock level, an account balance, an order status or an entitlement, “as of whenever” is not a degraded answer, it is a false one.

There is a related shape worth recognising. Some corpora are mostly stable with a thin volatile layer: product descriptions that rarely change, prices that change daily. Splitting those, indexing the stable part and looking up the volatile part, is usually right and is invisible if you think of the corpus as one thing.

What Grounding Buys, and What It Does Not

Anthropic documents a citations capability that grounds responses in supplied source documents and hands back the precise passages that support each statement, so the output can be checked against what it was built from.[1]

That is worth having and it is worth being precise about what it gives you. It **makes an answer checkable. It does not make it correct**. The passage returned genuinely supports the claim; whether the passage itself is current, authoritative or the right document is a question the capability cannot answer. Grounding against a stale corpus produces a verifiable route to the wrong answer, which is why the previous section matters more than this one.

What it does buy is real. A human reading the output can follow any statement back to the text it rests on, which turns an argument about whether the system is right into an inspection of one specific paragraph. In a scenario weighing two designs that both stay inside a governed corpus, the design that lets a reader **trace each statement back to the passage behind it** is doing something the **other simply cannot**, and that difference usually settles the question.

One economic note that belongs with this. The Files API is a create-once, use-many arrangement: upload a file once and reference it by identifier in later requests, instead of re-uploading it with each one.[2] When a scenario describes the same documents being sent again on every request, the **per-request cost is growing with the corpus** and it **does not need** to.

A Handle You Handed Out Can Go Stale Silently

One documented behaviour generalises well past the tool it describes, and it is the cleanest example in the documentation of a live-state hazard.

When the model is given element references for a browser tab, those references are scoped to that tab and stay valid only until the tab navigates or its DOM changes materially. Anthropic is explicit that the API cannot detect a stale or unknown reference, so the executor must **return an error result** telling the model the **reference is stale**, and warns against renumbering references already handed out for a tab, because renumbering silently invalidates references the model is still holding.[3]

Three transferable rules sit in that paragraph.

A **handle** is a **claim about the world at the moment it was issued**. Anything you hand the model that points at external state, an element reference, a row identifier, a cached result, a session token, is a snapshot with an expiry nobody wrote down.

When the **platform cannot detect staleness, your executor is the only thing** that can. Silence gets interpreted as validity.

Renumbering is worse than invalidating. An identifier that now points at something different produces a confidently wrong action, where an identifier that fails produces an error you can handle. **Reuse of identifiers is the quiet version of this mistake**, and it turns up in caches and row keys far more often than in browser tabs.

How Wrong Answers Are Built on Retrieval Questions

- **One strategy imposed on two query shapes**. The option picks semantic or lexical matching for all traffic and describes it as simplification. The test: does the described traffic contain **both identifiers and concept questions**? If it does, the **answer routes**.

- A **cadence answer to an architectural problem**. The option rebuilds the index more often, or adds a freshness field, against data that moves continuously. The test: does the data **change faster than any rebuild schedule** you would actually run?

- **Grounding treated as correctness**. The option offers citations, source attribution or quoting as the fix for wrong answers. The test: would this have **caught the error**, or only **shown you where it came** from?
- **One failure mode named when two are present**. The option correctly identifies a real defect and ignores a second one in the same scenario. The test: **read the scenario again for a second symptom** before choosing.

When the scenario says	The answer is usually	The trap is usually
Users search by part number, order ID or error code	Exact or lexical matching, or a keyed lookup	Semantic search, which blurs identifiers into their neighbours
Users ask questions in their own words about how something works	Semantic matching over embedded chunks	Keyword search, which misses the vocabulary mismatch entirely
Both kinds of query arrive on the same system	Keep both strategies and route each query to the right one	Choosing one strategy and calling it a simplification
The data changes between indexing and asking	Call the system that owns the fact	Rebuilding the index more often
The answer must be checkable by a reader	Return the supporting passage alongside the claim	Assuming attribution makes the underlying answer correct
The same documents are re-sent on every request	Upload once and reference by identifier	Letting per-request cost grow with the corpus

When the scenario says	The answer is usually	The trap is usually
The model holds a reference to something that may have changed	Return an explicit error saying the reference is stale	Silence, which reads as still valid
Identifiers might be reassigned or renumbered	Never reuse; let the old one fail	Renumbering, which turns a detectable error into a confident wrong action

The habit worth carrying: for **every fact the system returns, ask when it was true**. Retrieval architectures fail on time far more often than they fail on relevance.

What to Remember

- A corpus usually carries **two query shapes with opposite requirements**. Identifier lookups need **exact matching** because embedding **discards surface form** and blurs one code into its neighbours; conceptual questions need **semantic matching** because the **matching words are not present** in the text. Traffic containing both is answered by **keeping both strategies and routing between** them, never by choosing one.

- **Retrieval applied to live state** is pointing an index at data that **changes faster than the index is rebuilt**, and it produces the worst available output: a **confident, well-sourced, stale answer**. No rebuild cadence repairs it, because the defect is architectural.

- The system of record answers as of **now**, an index answers as of **whenever**. For stock levels, balances, order status and entitlements, the second is not a degraded answer, it is a **false one**.

- A corpus that is **mostly stable with a thin volatile layer** should be split, **indexing the stable part** and **looking up the volatile part**.

- **Grounding** returns the exact passages supporting each claim, which makes an answer **checkable rather than correct**. It cannot tell you whether the

passage is **current or authoritative**, so grounding against a stale corpus gives you a **verifiable route to a wrong answer**.

- **Uploading a file once and referencing it by identifier** replaces re-sending it with every request. When per-request cost **grows with the size of the corpus**, that is the thing to fix.
- Any handle you give the model is a **snapshot with an unwritten expiry**. Browser element references stay valid only until the tab **navigates or its DOM changes materially**, and the **API cannot detect a stale reference**, so the executor must **return an explicit error** saying so. Silence is **read as validity**.
- **Never renumber or reuse an identifier** already handed out. A reused identifier produces a **confidently wrong action**; an identifier that simply fails produces an **error you can handle**.